Universidad EIA · Fundamentos de Inteligencia Artificial · 2026-2



Predicción de abandono de clientes de telecomunicaciones

Clasificación de clientes con el conjunto Iranian Churn: comparación de 12 modelos con 4 estrategias de balanceo, ajuste de hiperparámetros, calibración de probabilidades y aplicación web.

Juan Jose Jaramillo Mora
Sebastian Giraldo Franco
Martin Restrepo
Miguel Angel Zuleta
Julian Mora
Santiago Zuluaga

Informe final
15 de octubre de 2026

Resumen

Este trabajo aborda la predicción del abandono de clientes de una empresa de telecomunicaciones a partir de nueve meses de uso del servicio. Usamos el conjunto *Iranian Churn* de UCI, con 3.150 clientes, de los cuales el 15,71 % abandonó en el mes doce. Es una clasificación binaria con desbalance moderado: un modelo que siempre responda «permanece» alcanza una exactitud de 84,29 % sin detectar ni un solo abandono. Por eso la métrica principal es el F1 de la clase que abandona.

Todo el análisis vive en un único cuaderno, organizado como las doce etapas de un proyecto de analítica (del entendimiento del negocio al despliegue). Se partió de trece variables de entrada y solo se eliminó `edad`, por ser idéntica a `grupo_edad`. Se conservaron `estado` y `valor_cliente` después de comprobar que aportan información, aunque su definición exacta no esté documentada por UCI. Se separó una muestra de prueba del 20 % con partición estratificada simple, se decidió el escalado (`MinMaxScaler`) y se compararon 12 clasificadores con 4 estrategias de balanceo (41 combinaciones) mediante validación cruzada estratificada de 10 particiones.

El modelo final es un Random Forest de 400 árboles con pesos de clase balanceados, probabilidades calibradas con una curva sigmoide y un umbral de decisión de 0,35. En validación cruzada el F1 es de 0,8722; en la muestra de prueba, de 0,9029, con 93 de 99 abandonos detectados y 14 falsas alarmas entre 531 clientes que permanecen. Como la prueba tiene pocos abandonos, el resultado se reporta como un rango, entre 0,87 y 0,90, y no como una cifra exacta. Concluimos que el modelo sirve para ordenar clientes por riesgo y priorizar contactos de retención, no como un sistema que dé certeza ni que prometa un ahorro.

Palabras clave: abandono de clientes, telecomunicaciones, clasificación desbalanceada, F1, PCA, calibración de probabilidades, scikit-learn.

Contenido

1 Introducción

2 El conjunto de datos

3 Partición de entrenamiento y prueba

4 Análisis exploratorio

5 Preparación: escalado, selección de variables y PCA

6 Modelado: comparación de 41 combinaciones

7 Ajuste de hiperparámetros

8 Calibración y umbral

9 Evaluación final

10 Despliegue: importancia de variables y aplicación web

11 Conclusiones, limitaciones y trabajo futuro

12 Referencias

A Anexo: reproducibilidad y estructura del repositorio

1. Introducción

Una empresa de telecomunicaciones en Irán pierde clientes frente a la competencia. Conseguir un cliente nuevo cuesta bastante más que retener a uno que ya se tiene, así que a la empresa le interesa saber quién está a punto de irse para contactarlo antes. El equipo de retención no puede llamar a todos los clientes: tiene capacidad para atender a unos pocos cada mes, así que lo que necesita es una lista corta de clientes en riesgo, ordenada por prioridad.

Los dos errores posibles no cuestan igual: dejar escapar a un cliente que sí se iba significa perderlo; llamar de más a uno que no pensaba irse solo gasta una llamada. Por eso el proyecto prioriza no dejar escapar clientes (recall) sin descuidar del todo acertar en las llamadas (precisión).

El problema se plantea como una clasificación binaria: para cada cliente se predice si abandona (1) o permanece (0). La métrica principal es el F1 de la clase abandono, que combina precisión y recall; no se usa la exactitud para decidir nada, porque con 84,29 % de clientes que permanecen, un modelo que siempre responda «permanece» ya saca una exactitud muy alta sin detectar a nadie en riesgo. El modelo además debe entregar una probabilidad por cliente, no solo una respuesta de sí o no, para poder ordenar la lista de contactos por prioridad.

Para anticipar el abandono se necesita información de cada cliente anterior al momento en que se va. Se usa el conjunto *Iranian Churn* de UCI, cuyas entradas corresponden a los primeros nueve meses de cada cliente, mientras que el abandono se registra al terminar el mes doce. El archivo no trae identificador de cliente ni fechas de cada medición: sin identificador no se puede saber si dos filas iguales son la misma persona o dos distintas, y sin fechas no se puede comprobar el orden exacto de los hechos. Estas dos limitaciones se tienen en cuenta en varias decisiones posteriores y se retoman en las limitaciones del trabajo (sección 11.2).

Todo el análisis vive en un único cuaderno reproducible, organizado como las doce etapas de un proyecto de analítica (entendimiento del negocio, enfoque analítico, requerimiento de datos, recolección, separación de entrenamiento y prueba, entendimiento de los datos, preparación, modelado, ajuste de hiperparámetros, calibración y umbral, evaluación final y despliegue), en vez de como secciones técnicas sueltas. Este informe sigue el mismo orden y describe además la aplicación web que publica el modelo.

2. El conjunto de datos

Se usa *Iranian Churn Dataset*, publicado en el repositorio de aprendizaje automático de UCI (identificador 563, licencia CC BY 4.0). Los datos provienen de una empresa de telecomunicaciones iraní y resumen, para cada cliente, el uso del servicio durante nueve meses junto con si abandonó o permaneció en el mes doce. El archivo original tiene 3.150 filas y 14 columnas (13 variables más la respuesta), sin valores faltantes. El archivo no se modifica en ningún momento; todo lo derivado se guarda con otro nombre.

Se encontraron 300 filas repetidas exactamente. Se conservan: sin identificador de cliente no se puede afirmar que sean errores de captura y no clientes distintos con el mismo comportamiento, así que cada fila se trata como un cliente distinto.

Tabla 1. Variables del conjunto de datos, en el orden en que se revisan.

Variable	Descripción	Tipo
`fallas_llamadas`	Llamadas fallidas	Numérica (conteo)
`quejas`	0 sin queja; 1 con queja (no cuenta cuántas)	Binaria
`antiguedad`	Meses de suscripción	Numérica
`nivel_cargo`	Nivel de cargo de 0 a 9 (hay valores de 10 en los datos)	Ordinal
`segundos_uso`	Segundos de llamadas	Numérica
`frecuencia_uso`	Número de llamadas	Numérica (conteo)
`frecuencia_sms`	Número de mensajes	Numérica (conteo)
`numeros_distintos`	Números distintos llamados	Numérica (conteo)
`grupo_edad`	Grupo de edad, de 1 a 5	Ordinal
`plan_tarifario`	1 pago por uso; 2 contractual	Nominal binaria
`estado`	1 línea activa; 2 no activa	Nominal binaria
`edad`	Edad; se elimina en la sección 4 por ser redundante con `grupo_edad`	Numérica
`valor_cliente`	Valor calculado por la empresa a partir de otras variables; UCI no publica su fórmula	Numérica derivada
`abandono`	1 si el cliente abandonó en el mes doce; 0 si permaneció	Objetivo

`estado` y `valor_cliente` merecen una revisión aparte, porque de la primera no se sabe con precisión qué cuenta como «activo» y de la segunda no se conoce la fórmula. Esa revisión, hecha ya con los datos de entrenamiento separados, se describe en la sección 4.6.

La clase objetivo está desbalanceada: 2.655 clientes permanecieron y 495 abandonaron, es decir, 84,29 % frente a 15,71 %.

3. Partición de entrenamiento y prueba

Se separa un 20 % de prueba con `train_test_split`, estratificado por la variable objetivo y con semilla fija (`random_state=42`), **antes** de cualquier análisis exploratorio. La partición se hace sobre las trece variables de entrada originales (todavía con `edad`, que se elimina después en la sección 4.6, igual en los dos conjuntos) más la respuesta.

Tabla 2. Tamaño y desbalance de cada conjunto.

Conjunto	Filas	Permanece (0)	Abandona (1)	% abandono
Completo	3.150	2.655	495	15,71 %
Entrenamiento	2.520	2.124	396	15,71 %
Prueba	630	531	99	15,71 %

Los tres coinciden en 15,71 % de abandono, gracias a la estratificación. A diferencia de un esquema con validación cruzada por grupos, aquí no se agrupan perfiles idénticos antes de partir: cada fila se trata como un cliente independiente (sección 2), así que puede haber el mismo perfil de entrada repartido entre entrenamiento y prueba. No se mide ni se reporta ese solapamiento; se deja como limitación explícita (sección 11.2).

> Regla que no se rompe en el resto del trabajo: desde aquí, toda decisión —exploración, preparación, modelado, ajuste, calibración y umbral— se toma únicamente con las 2.520 filas de entrenamiento. Los 630 clientes de prueba no se vuelven a tocar hasta la sección 9, una sola vez.

4. Análisis exploratorio

4.1 El desbalance

En entrenamiento, 2.124 clientes permanecieron y 396 abandonaron (15,71 %). Un modelo que siempre respondiera «permanece» acertaría la mayoría de las veces sin detectar a ninguno en riesgo: la razón de fondo para que la exactitud nunca decida en este proyecto.

4.2 Variables categóricas

Dos variables separan los grupos con claridad: abandona el 80,69 % de los 202 clientes con queja, frente al 10,05 % del resto; y el 47,44 % de los 624 clientes con línea no activa, frente al 5,27 % de los activos. El plan contractual abandona un 3,05 % frente al 16,79 % del plan de pago por uso. En `nivel_cargo` el abandono baja al subir el nivel y de 5 en adelante no hay ninguno, pero esos niveles tienen entre 6 y 24 clientes, así que no es una conclusión fuerte con tan poca muestra.

4.3 Variables numéricas

Las ocho variables numéricas tienen la media por encima de la mediana, señal de una cola larga a la derecha: las escalas también son muy distintas entre sí, desde `antiguedad` entre 3 y 47 hasta `segundos_uso`, que llega a 16.980 — el escalado importará en la sección 5.

Quien abandona usa mucho menos el servicio: la mediana de `segundos_uso` es 1.188 frente a 3.594 entre quienes permanecen, la de `frecuencia_uso` es 25 frente a 64, la de `numeros_distintos` es 10 frente a 23 y la de `valor_cliente` es 97,82 frente a 267,94. `antiguedad` y `edad` dan casi igual en los dos grupos: la señal está en el uso, no en quién es el cliente.

4.4 Asociación de cada variable con el abandono (phik)

Se mide la relación entre variables con el coeficiente phik (φk), que funciona igual de bien para variables numéricas y categóricas, a diferencia de la correlación de Pearson, que solo sirve para números.

Tabla 3. Coeficiente phik de cada variable con el abandono, entrenamiento.

Variable	phik
`quejas`	**0,734**
`estado`	**0,705**
`segundos_uso`	**0,469**
`frecuencia_uso`	**0,445**
`valor_cliente`	**0,413**
`numeros_distintos`	**0,389**
`frecuencia_sms`	**0,325**
`plan_tarifario`	**0,152**
`grupo_edad` / `edad`	**0,105**
`fallas_llamadas`	**0,066**

4.5 Variables que repiten información

Hay 19 parejas de variables con phik de 0,60 o más. El caso extremo es `edad` y `grupo_edad`, con phik de 1,000: cada edad pertenece a un único grupo y cada grupo tiene una sola edad ($15 \leftrightarrow 1$, $25 \leftrightarrow 2$, $30 \leftrightarrow 3$, $45 \leftrightarrow 4$, $55 \leftrightarrow 5$), así que son la misma información escrita de dos maneras. Se elimina `edad` y se conserva `grupo_edad` por ser la forma documentada por UCI. Es la única variable que se elimina en todo el proyecto; quedan doce entradas más la respuesta. Las variables de uso también se solapan bastante entre sí (`segundos_uso` con `frecuencia_uso`, 0,928; `frecuencia_sms` con `valor_cliente`, 0,918), pero cada una mide algo distinto y se conservan todas.

4.6 Estado y valor del cliente

Antes de confiar en estas dos variables dudosas se comprueba que de verdad aportan algo. De `estado` se revisa si distingue por su cuenta o solo repite lo que ya dicen las quejas: incluso entre los 2.318 clientes que nunca presentaron ninguna queja, el abandono varía bastante según el estado, así que aporta información adicional. De `valor_cliente` se mide si es una combinación de las demás entradas con una regresión que intenta reconstruirlo: las otras once explican el 98 % de su variación, es decir, es casi una combinación matemática de las demás. Aun así se conserva: no introduce información de fuera del periodo medido, y en la sección 10.1 resulta ser la sexta variable más usada por el modelo final.

> Queda un conjunto de doce entradas más `abandono`. Las señales más fuertes por phik son `quejas` y `estado`; el bloque de uso viene después; el perfil del cliente (edad, plan) casi no distingue nada por sí solo.

5. Preparación: escalado, selección de variables y PCA

5.1 Escalado

Las variables están en escalas muy distintas y sus distribuciones están sesgadas (sección 4.3). Los modelos que miden distancias entre clientes (KNN, SVM, redes neuronales) quedarían dominados por las variables de números grandes si no se escala; a los modelos de árbol no les afecta, porque cortan cada variable por separado. Se usa `MinMaxScaler`, que lleva cada variable al rango de 0 a 1: conserva la forma de la distribución y deja los ceros en cero, que aquí significan «cliente sin uso». El escalador se ajusta siempre con el entrenamiento y se aplica a los dos conjuntos, nunca al revés; en la comparación de modelos (sección 6) esto se repite dentro de cada una de las 10 particiones de validación cruzada.

5.2 Selección de variables con ANOVA

ANOVA compara, para cada variable, el promedio de quienes abandonan con el de quienes permanecen. Un valor de F alto indica que los dos grupos están lejos; un p pequeño, que esa distancia difícilmente se explica por azar.

Tabla 4. ANOVA (`f_classif`) de cada variable con el abandono, entrenamiento.

Variable	F	p
`quejas`	**968,57**	**0,0000**
`estado`	**839,51**	**0,0000**
`frecuencia_uso`	**259,48**	**0,0000**
`segundos_uso`	**252,24**	**0,0000**
`valor_cliente`	**229,31**	**0,0000**
`numeros_distintos`	**212,87**	**0,0000**
`frecuencia_sms`	**128,76**	**0,0000**
`nivel_cargo`	**108,20**	**0,0000**
`plan_tarifario`	**26,14**	**0,0000**
`antiguedad`	**4,49**	**0,0343**
`fallas_llamadas`	**0,68**	**0,4091**
`grupo_edad`	**0,11**	**0,7444**

El orden coincide con el de phik. Al final quedan dos variables sin relación individual aparente, `fallas_llamadas` y `grupo_edad` ($p > 0{,}05$). ANOVA mira cada variable sola: una F baja no demuestra que la variable sea inútil, puede aportar combinada con otras. Esto se comprueba de verdad con los modelos reales en la sección 6.4, y no al revés.

5.3 Análisis de componentes principales (PCA)

PCA se aplicó también, como alternativa a la selección de variables. A diferencia de ANOVA, que elige un subconjunto de las variables *originales* tal cual, PCA construye variables nuevas (componentes), cada una una combinación lineal de las doce originales, ordenadas de mayor a menor varianza explicada. Se escaló primero con `MinMaxScaler` (ajustado con entrenamiento), porque las componentes se construyen buscando la mayor variación y, sin escalar, las variables de números grandes dominarían el resultado solo por tener magnitudes mayores, no por distinguir mejor a los clientes.

Tabla 5. Varianza acumulada por número de componentes de PCA.

Componentes	% varianza acumulada
1	38,11 %
2	52,74 %
4	74,43 %
6	89,37 %
7	93,68 % (pasa el 90 %)
8	96,70 % (pasa el 95 %)
12	100 %

La compresión es escasa: hacen falta 7 de las 12 variables originales para conservar el 90 % de la varianza y 8 para el 95 %. Se descartó PCA por dos razones. Primera, casi no comprime: pasar de doce variables a siete u ocho componentes no es una reducción que valga la pena, y el problema no tenía de entrada un problema de demasiadas variables. Segunda, y más importante: cada componente es una mezcla de las doce variables originales, así que un modelo entrenado sobre componentes no se podría explicar variable por variable. Si el equipo de retención pregunta por qué un cliente está en riesgo, hace falta poder responder «por sus pocas llamadas y su queja reciente», no «por la componente 3» — algo que no significa nada para una persona. Esa necesidad de explicación es la misma razón con la que, en la sección 7.3, se prefiere Random Forest sobre modelos menos interpretables ante un empate.

5.4 Decisión

La preparación queda en escalado con `MinMaxScaler`, sin PCA y sin descartar variables todavía por ANOVA: esa decisión se comprueba con los modelos reales en la sección 6.4, no se toma a priori.

6. Modelado: comparación de 41 combinaciones

6.1 Los doce modelos y las cuatro estrategias de balanceo

Se comparan los doce clasificadores vistos en el curso: ocho individuales (KNN, Naive Bayes, árbol de decisión, regresión logística, SVM, SVM lineal, red neuronal MLP y Random Forest) y cuatro ensambles (Bagging, AdaBoost, Gradient Boosting y Stacking, que combina Random Forest, SVM y KNN y decide entre ellos con una regresión logística). Se prueban cuatro formas de tratar el desbalance: sin balanceo (referencia), peso por clase (`class_weight="balanced"`, solo en cinco modelos que admiten ese parámetro), SMOTENC (crea clientes sintéticos de la clase minoritaria interpolando entre los reales, respetando que `quejas`, `plan_tarifario` y `estado` son categorías) y submuestreo aleatorio (descarta clientes de la mayoría hasta igualar los dos grupos). Con doce modelos por tres estrategias que todos admiten, más cinco de peso por clase, salen **41 combinaciones**.

6.2 Cómo se evita la fuga de información

Para cada una de las 10 particiones de validación cruzada estratificada: se separa la parte de entrenamiento de la que se evalúa; el escalador se ajusta solo con la parte de entrenamiento de esa vuelta y se aplica a las dos; y el balanceo (SMOTENC o submuestreo) se aplica únicamente a la parte de entrenamiento. Así el modelo nunca ve nada de la parte con la que se mide, ni siquiera a través del escalado o de clientes sintéticos construidos con información de la parte de evaluación.

6.3 Resultados de las 41 combinaciones

Tabla 6. Las ocho mejores de las 41 combinaciones, por F1 en validación cruzada.

Modelo	Estrategia	F1	Desv. F1	Precisión
Stacking	SMOTENC	0,8717	0,0394	0,8586
Random Forest	Peso por clase	0,8717	0,0399	0,8514
Random Forest	SMOTENC	0,8660	0,0399	0,8573
Stacking	Sin balanceo	0,8603	0,0285	0,8926
Red neuronal	Sin balanceo	0,8498	0,0318	0,8756
Red neuronal	SMOTENC	0,8441	0,0324	0,7675
Bagging	SMOTENC	0,8343	0,0539	0,8297
Random Forest	Sin balanceo	0,8338	0,0341	0,8872

Los tres primeros están a 5 milésimas de diferencia, mucho menos que la desviación entre particiones (≈0,035): no hay un ganador claro todavía. La separación grande está entre familias de modelos: árboles y conjuntos pasan de 0,80; regresión logística, los dos SVM y

Naive Bayes se quedan por debajo de 0,70. Con doce variables y relaciones que no son líneas rectas, los modelos que parten el espacio en regiones aprovechan mejor los datos: entre quienes se quejaron y además usaron poco el servicio, el abandono llega a cerca del 90-100 % según el cruce exacto, mientras que entre quienes no se quejaron y usaron mucho baja a 2 %. Un modelo lineal suma el efecto de cada variable por separado y no puede expresar directamente «se quejó y usó poco»; un árbol sí, porque puede partir primero por quejas y luego por uso.

Tabla 7. F1 promedio y mejor F1 por estrategia de balanceo.

Estrategia	Combinaciones	F1 promedio	Mejor F1	Recall promedio
Sin balanceo	12	0,7218	0,8603	0,6945
Peso por clase	5	0,7301	0,8717	0,8874
SMOTENC	12	0,7475	0,8717	0,8921
Submuestreo	12	0,7016	0,7772	0,9240

El submuestreo tiene el mejor recall promedio (0,9240) y el peor F1: detecta casi todos los abandonos a costa de muchas falsas alarmas, porque para igualar los dos grupos hay que descartar más del 80 % de los clientes que permanecen y el modelo se queda con pocos datos. El balanceo pesa distinto según el modelo: la regresión logística mejora bastante (0,5395 a 0,6582), Random Forest mejora poco (0,8338 a 0,8717), porque ya manejaba el desbalance razonablemente bien.

Sin balancear, varios modelos rondan el 93-95 % de exactitud casi sin detectar abandonos, muy cerca del 84,29 % de responder siempre «permanece» (F1 de 0): guiarse por la exactitud habría llevado a preferir justo los modelos menos útiles.

6.4 Los tres candidatos y si conviene quitar variables

Se toman las tres mejores combinaciones por F1, sin repetir modelo, para tener candidatos de familias distintas: Random Forest con peso por clase (0,8717), Stacking con SMOTENC (0,8717) y la red neuronal sin balancear (0,8498). Con estos tres candidatos se comprueba de verdad si conviene quitar variables con ANOVA (la pregunta que quedó abierta en la sección 5.2), entrenando cada uno con las doce variables completas, con las diez mejores y con las ocho mejores.

Tabla 8. F1 de los tres candidatos según el número de variables.

Modelo	12 variables	10 variables	8 variables
Stacking	**0,8717**	0,8290	0,7941
Random Forest	**0,8717**	0,8181	0,7837
Red neuronal	**0,8498**	0,7734	0,6905

Quitar variables empeora a los tres candidatos, y no por poco (la red neuronal cae de 0,85 a 0,69). Las dos variables que ANOVA dejaba últimas, `grupo_edad` y `fallas_llamadas`, son las primeras que se descartan al bajar a diez variables, y perderlas cuesta más de cinco centésimas de F1: por separado no distinguían a quienes abandonan, pero combinadas con las demás sí aportan. Se conservan las **doce variables completas**, sin selección.

7. Ajuste de hiperparámetros

7.1 Por qué el ajuste va con `Pipeline`

`GridSearchCV` entrena directamente el modelo que se le entrega, con los datos que uno le pasa: entre partir los datos y entrenar no hay dónde meter el escalado ni el balanceo. Si se aplicaran antes, sobre todo el entrenamiento, aprenderían también de las filas que después se usan para evaluar. Con SMOTENC el problema es mayor, porque crea clientes nuevos interpolando entre los que abandonaron: si se aplicara antes de partir en pliegues, algunos sintéticos saldrían de clientes que después se evalúan, y además las particiones de evaluación quedarían con una proporción de abandono irreal. La solución es entregarle a `GridSearchCV` un `Pipeline` de imbalanced-learn (Lemaître et al., 2017) que lleve los pasos adentro: al entrenar, el escalado y el balanceo trabajan solo con la parte de entrenamiento de esa vuelta; al evaluar, el escalador aplica lo que ya aprendió y el balanceo no hace nada.

7.2 Las tres búsquedas

Tabla 9. Ajuste de hiperparámetros de los tres candidatos.

Modelo	Qué se varió	Mejor configuración	F1 ajustado
Random Forest	Nº de árboles, profundidad, mínimo por hoja	400 árboles, sin límite de profundidad, hoja mínima 1	0,8722
Stacking	Árboles del RF interno, vecinos del KNN interno	200 árboles, 5 vecinos	0,8717
Red neuronal	Forma de las capas ocultas, alpha	capas (100, 50), alpha 0,0001	0,8721

Todas se buscan con `GridSearchCV(scoring="f1")` y las mismas 10 particiones de la sección 6. A la red neuronal se le subió el máximo de iteraciones a 2.000, porque en la sección 6 varias particiones no habían terminado de converger con 1.000.

7.3 El modelo elegido

Los tres candidatos ajustados quedaron a milésimas de F1, muy por debajo de la desviación entre particiones (≈0,03-0,04). Elegir el número más alto sería casi azar: con otra semilla de particiones el orden cambiaría. Se aplica en su lugar una regla conocida: entre las configuraciones que caen dentro de un error estándar de la mejor (la desviación entre particiones dividida por la raíz de 10), se elige la más rápida de entrenar.

Tabla 10. F1, desviación y costo de entrenamiento de los tres candidatos ajustados.

Modelo	F1	Desviación	Error estándar
Random Forest	**0,8722**	**0,0326**	**0,0103**
Red neuronal	0,8721	0,0348	0,0110
Stacking	0,8717	0,0374	0,0118

Los tres caen dentro del margen de un error estándar de la mejor, así que se elige el más rápido de entrenar: **Random Forest**. Además de la velocidad, pesan otras tres razones: es un solo modelo, mientras Stacking entrena tres clasificadores más una regresión logística que decide entre ellos; permite saber qué variables pesaron en cada predicción (sección 10.1), lo que el equipo de retención necesita para preparar la llamada — el mismo argumento con el que se descartó PCA en la sección 5.3; y con peso por clase no hay remuestreo, así que no hay clientes sintéticos que explicar ni decisiones sobre qué variables son categóricas para SMOTENC.

> Modelo elegido: Random Forest, 400 árboles, sin límite de profundidad, hoja mínima 1, peso por clase (`class_weight="balanced"`), dentro de un `Pipeline` con `MinMaxScaler`.

8. Calibración y umbral

El modelo no solo dice sí o no: entrega una probabilidad de abandono. De esa probabilidad dependen dos cosas. La primera es el orden de la lista que recibe el equipo de retención, y para eso la probabilidad debe querer decir lo que dice: si el modelo asigna 30 % a un grupo de clientes, deberían abandonar cerca del 30 % de ellos — eso es estar calibrado. La segunda es a partir de qué probabilidad se da la alarma; 0,5 es el valor por defecto, pero no tiene nada de especial, y con clases desbalanceadas casi nunca es el que da el mejor F1. Todo se decide con entrenamiento; la prueba sigue sin abrirse.

8.1 Probabilidades fuera de pliegue

Se necesitan probabilidades que el modelo haya dado sin haber visto a ese cliente al entrenar. Se usa `cross_val_predict` con las mismas diez particiones: cada uno de los 2.520 clientes recibe la probabilidad que le asignó un modelo entrenado sin él. El promedio de esas probabilidades es 0,1865, por encima del 0,1571 de abandono real — el modelo exagera el riesgo en promedio, esperable porque el peso por clase lo empuja a favor de la minoría.

8.2 Calibración sigmoide

`CalibratedClassifierCV` (Platt, 1999) aprende una función que corrige las probabilidades del modelo, ajustada también por particiones para no usar los mismos datos con los que se entrenó.

Tabla 11. Puntaje de Brier antes y después de calibrar (más bajo es mejor).

	Brier
Sin calibrar	0,0368
Calibrado (sigmoide)	0,0336

El puntaje de Brier baja de 0,0368 a 0,0336. La mejora es modesta porque Random Forest ya ordenaba bien a los clientes: lo que corrige la calibración es sobre todo el sesgo hacia arriba que mete el peso por clase, no el orden de la lista. Se usan las probabilidades calibradas de aquí en adelante.

8.3 El umbral

Se recorren los umbrales de 0,05 a 0,95 y se mide el F1, la precisión y el recall de cada uno sobre las probabilidades fuera de pliegue.

Tabla 12. F1, precisión y recall según el umbral, sobre entrenamiento.

Umbral	F1	Precisión	Recall	Alertas (de 2.520)
0,05	0,7524	0,6097	0,9823	638
0,20	0,8624	0,7953	0,9419	469
0,30	0,8658	0,8306	0,9040	431
0,35	0,8713	0,8544	0,8889	412
0,40	0,8672	0,8607	0,8737	402
0,50	0,8590	0,8724	0,8460	384
0,70	0,8116	0,8988	0,7399	—

El F1 más alto está en el umbral 0,35, con 0,8713. La curva es bastante plana entre 0,25 y 0,40, todos por encima de 0,86, así que moverse dentro de ese rango cambia poco. El intercambio se ve con claridad en los extremos: con umbral 0,05 el recall es 0,9823 pero la precisión 0,6097, de 638 alertas casi la mitad serían clientes que no pensaban irse; con 0,70 la precisión sube a 0,8988 y el recall cae a 0,7399, se escaparía más de la mitad de los que sí se van.

> Umbral elegido: 0,35. Frente al 0,50 por defecto, el F1 sube de 0,8590 a 0,8713. Ese corte alertaría sobre 412 de los 2.520 clientes de entrenamiento (16,4 %); si el equipo de retención tuviera menos capacidad, el umbral es la perilla para ajustar cuántas alertas salen, a cambio de dejar escapar más clientes.

9. Evaluación final

Se abren los 630 clientes de prueba reservados en la sección 3, que no han participado en ninguna decisión: ni en la exploración, ni en la comparación de modelos, ni en el ajuste, ni en la elección del umbral. Modelo, calibración y umbral ya están congelados. Esta medición se hace una sola vez: si después de verla se volviera a cambiar el modelo o el umbral, la prueba dejaría de ser independiente y el número perdería su sentido.

Se entrena con las 2.520 filas de entrenamiento completas el modelo elegido en la sección 7, con la calibración de la sección 8, y se predice sobre los 630 de prueba con el umbral 0,35.

Tabla 13. Métricas del modelo final en la muestra de prueba (630 clientes, 99 abandonos).

Métrica	Valor
F1 del abandono	0,9029
Recall	0,9394
Precisión	0,8692
Exactitud	0,9683
ROC AUC	0,9861
Precisión promedio	0,9208
Brier	0,0278

El F1 de prueba (0,9029) queda por encima del 0,8722 de validación, y no debe leerse como que el modelo resultó mejor de lo esperado: se entrena con 2.520 filas en vez de las ≈2.268 de cada vuelta de validación, y sobre todo, 630 clientes con 99 abandonos son pocos, así que esta cifra tiene su propio margen, de unas tres centésimas. Lo razonable es decir que las dos mediciones son consistentes y que el desempeño está entre 0,87 y 0,90. La exactitud (0,9683) es la métrica que se decidió no usar para decidir, y aquí se ve por qué: un modelo que respondiera siempre «permanece» sacaría 0,8429 sin detectar a nadie.

Tabla 14. Matriz de confusión en la muestra de prueba.

	El modelo alerta	El modelo no alerta
Abandonó (real)	93 detectados	6 no detectados
Permaneció (real)	14 falsas alarmas	517 bien descartados

De los 99 clientes que abandonaron, el modelo detectó 93 y dejó escapar 6. De los 531 que permanecieron, alertó de más sobre 14. En términos de la operación, la lista tendría 107 nombres, 93 de ellos clientes que de verdad se iban; el costo son 14 llamadas gastadas en gente que no pensaba irse, y el riesgo que queda son 6 clientes que se pierden sin aviso.

9.1 Priorizar contactos

Para el uso previsto —decidir a quién llamar primero— importa más el orden por riesgo que el umbral exacto. Se ordenan los 630 clientes de prueba por probabilidad y se mide cuántos abandonos reales caen en los primeros puestos.

Tabla 15. Qué pasa si solo se contacta a una fracción de los clientes de mayor riesgo.

% contactado	Clientes	Precisión (aciertos)	Recall (abandonos cubiertos)	Veces mejor que al azar
5 %	32	96,88 %	31,31 %	6,16×
10 %	63	93,65 %	59,60 %	5,96×
15 %	94	90,43 %	85,86 %	5,75×
20 %	126	75,40 %	95,96 %	4,80×
25 %	158	61,39 %	97,98 %	3,91×
30 %	189	51,32 %	97,98 %	3,27×

Contactando al 15 % de los clientes (94 personas) se alcanza al 85,86 % de los que abandonan y acierta el 90,43 % de las llamadas: 5,75 veces lo que daría llamar al azar. A partir del 20 % el rendimiento cae, porque se capturan nueve de cada diez abandonos pero ya solo tres de cada cuatro llamadas aciertan — las últimas llamadas rinden cada vez menos.

10. Despliegue: importancia de variables y aplicación web

10.1 En qué se apoya el modelo

Random Forest permite ver cuánto usó cada variable para separar a los clientes, sumando los 400 árboles. No dice si empuja hacia el abandono o en contra, solo cuánto se apoyó en ella.

Tabla 16. Importancia de cada variable en el modelo final.

Variable	Importancia
`estado`	**0,1687**
`segundos_uso`	**0,1411**
`frecuencia_uso`	**0,1392**
`quejas`	**0,1244**
`antiguedad`	**0,1007**
`valor_cliente`	**0,0949**
`numeros_distintos`	**0,0677**
`frecuencia_sms`	**0,0626**
`fallas_llamadas`	**0,0420**
`grupo_edad`	**0,0395**
`nivel_cargo`	**0,0177**
`plan_tarifario`	**0,0015**

Hay dos cosas que no coinciden con lo que mostró la exploración. `quejas` era la señal más fuerte por phik y por ANOVA, y aquí queda cuarta: es una variable binaria que solo afecta al 8 % de los clientes, así que el modelo la usa en pocos cortes, aunque sean muy decisivos. Y `antiguedad`, que por ANOVA casi no distinguía, aparece quinta porque el bosque la aprovecha combinada con las demás — otra forma de ver lo que ya mostró la sección 6.4. Al final quedan `nivel_cargo` y `plan_tarifario`, que el modelo prácticamente no usa.

10.2 Exportación del modelo

Se guardan tres cosas en un solo archivo: el modelo ya entrenado y calibrado (un único objeto, no un pipeline y un calibrador por separado), el umbral y los nombres de las doce columnas en el orden en que las espera. Aparte va un archivo de metadatos en JSON con las

decisiones y los resultados. Se comprobó que, al recargar el archivo guardado, el modelo predice exactamente las mismas probabilidades y las mismas predicciones sobre la prueba: el archivo reproduce el modelo.

10.3 Aplicación web

El modelo se publicó en una aplicación construida con Gradio (Abid et al., 2019), que carga el modelo exportado y reconstruye ella misma la partición 80/20 (misma semilla) a partir del archivo original, sin depender de archivos intermedios. Se ejecuta en local con `uv run python app.py` y está preparada para ejecutarse en Google Colab. Tiene tres pestañas:

- **Evaluar un cliente.** Un formulario con las doce entradas del modelo, cada una con su descripción. Muestra el riesgo estimado en porcentaje, un veredicto según el umbral de 0,35 y los factores que más empujan el riesgo de ese cliente en concreto, comparando cada entrada contra la mediana de entrenamiento. Botones de ejemplo cargan un cliente tranquilo, uno justo en el límite del umbral, uno en riesgo alto o uno real al azar de la muestra de prueba, mostrando en ese último caso qué ocurrió de verdad.
- **Priorizar contactos.** Recibe un archivo CSV con un cliente por fila (en español o con los nombres originales en inglés de UCI) y devuelve una tabla ordenada por riesgo, con un resumen y un archivo descargable.
- **Sobre el modelo.** Muestra el modelo, la estrategia de balanceo, la calibración, el umbral, las métricas de validación y de prueba, en qué se apoya el modelo (importancia de variables) y un deslizador en vivo que recalcula cuántos abandonos se cubren según cuántos clientes se contacten, sobre los 630 clientes reales de prueba.

La aplicación incluye un aviso visible de que es un proyecto académico, que no estima si una oferta concreta evitará el abandono ni cuánto ahorraría hacerlo, y que no reemplaza el criterio del equipo de retención.

11. Conclusiones, limitaciones y trabajo futuro

11.1 Conclusiones

- **El desbalance define la métrica.** Con 15,71 % de abandono, un modelo con F1 de 0 puede tener una exactitud de 84,29 %. La métrica principal es el F1 de la clase que abandona.
- **Casi todas las entradas originales se conservan.** De trece variables solo se elimina `edad`, por ser idéntica a `grupo_edad`. `estado` y `valor_cliente` se conservan después de comprobar que aportan, aunque su definición exacta no esté documentada.
- **La señal es fuerte y no es lineal.** Con phik de 0,734, las quejas son la variable más asociada al abandono, seguida de `estado` (0,705). La señal está en combinaciones de variables, por lo que los árboles y los ensambles superan con claridad a los modelos lineales (0,87 frente a 0,54-0,66 de F1).
- **PCA se probó y se descartó con evidencia.** Resumir las doce variables en componentes necesitaría siete de doce para conservar el 90 % de la varianza —casi no comprime— y haría imposible explicar el riesgo de un cliente variable por variable.
- **El modelo final es útil para priorizar contactos.** Random Forest con peso por clase, calibrado, detecta 93 de 99 abandonos de la muestra de prueba con solo 14 falsas alarmas entre 531 clientes que permanecen; contactando al 15 % de mayor riesgo se cubre el 86 % de los abandonos con 90 % de precisión.
- **El resultado se reporta como un rango.** F1 de validación 0,8722 y de prueba 0,9029; con 99 abandonos en la prueba, la diferencia cabe en el margen de la muestra, así que lo correcto es decir que el desempeño está entre 0,87 y 0,90, no una cifra exacta.

11.2 Limitaciones

- **Sin fechas ni identificador de cliente.** No se puede comprobar el orden exacto de los hechos cliente por cliente, ni descartar que alguna entrada (por ejemplo, uso igual a cero) ya reflejara un abandono ocurrido.
- **Perfiles posiblemente compartidos entre entrenamiento y prueba.** La partición no agrupa perfiles idénticos antes de partir (sección 3); no se midió cuántos clientes con el mismo perfil de entrada quedaron repartidos entre los dos conjuntos.
- **`estado`, la variable más usada por el modelo, es un indicador tardío.** Suele cambiar cuando el cliente ya dejó de usar el servicio, justo cuando queda menos margen para retenerlo.
- **Datos de una sola empresa.** No se sabe cómo se comportaría el modelo en otro periodo, otro mercado o con otra definición de abandono.
- **El umbral es una decisión de negocio.** El 0,35 maximiza el F1, pero el umbral adecuado depende de cuánto cueste contactar a un cliente frente a perderlo; ese costo no se estimó en este trabajo.

- **No se estima el efecto de una intervención.** El modelo ordena a los clientes por riesgo de abandono, no estima si una oferta de retención cambiaría ese riesgo ni cuánto ahorraría.

11.3 Trabajo futuro

- Conseguir datos con fecha para validar con una partición temporal y descartar por completo que alguna entrada refleje el desenlace.
- Medir el solapamiento de perfiles entre entrenamiento y prueba, y comparar contra una partición que agrupe perfiles idénticos.
- Costos asimétricos entre contactar de más y perder un cliente, y selección del umbral a partir de esos costos.
- Un diseño experimental (A/B) que mida si contactar a los clientes de mayor riesgo reduce el abandono real.
- Datos de otra empresa o periodo, y despliegue permanente de la aplicación.

---

12. Referencias

Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. y Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. *arXiv:1906.02569*.

Lemaître, G., Nogueira, F. y Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research, 18*(17), 1–5.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. En A. J. Smola et al. (Eds.), *Advances in Large Margin Classifiers*. MIT Press.

Iranian Churn Dataset (2020) [Conjunto de datos]. UCI Machine Learning Repository, identificador 563. https://doi.org/10.24432/C5JW3Z

Anexo. Reproducibilidad y estructura del repositorio

El entorno se gestiona con uv (`pyproject.toml` y `uv.lock`, Python 3.11) y el cuaderno también está pensado para correr en Google Colab. Para reproducir los resultados basta con clonar el repositorio, ejecutar `uv sync` y correr el cuaderno completo; no depende de ningún archivo intermedio, solo del CSV original de UCI.

Tabla A1. Archivos principales del repositorio.

Archivo	Responsabilidad	Genera
`Churn_Paso_a_Paso.ipynb`	Todo el análisis, de principio a fin: entendimiento del negocio, exploración, preparación, modelado, ajuste, calibración, evaluación y despliegue	`data/train.csv`, `data/test.csv`, `models/modelo_final.joblib`, `models/metadata_modelo.json`, `resultados/priorizacion_test.csv`
`app.py`	Aplicación Gradio que publica el modelo	—

El modelo entrenado (`models/modelo_final.joblib`) no se versiona en el repositorio: pesa cerca de 50 MB y se regenera ejecutando el cuaderno completo. Quien clona el repositorio lo obtiene corriendo `Churn_Paso_a_Paso.ipynb` antes de abrir la aplicación.